

Project Blueprint

Voice-Activated SRE (Agentic Infrastructure Supervisor)

1. Executive Summary

This project defines the architecture and implementation roadmap for a **Real-Time Agentic Voice Supervisor** specialized in infrastructure management. It transforms traditional MLOps and Kubernetes orchestration into a seamless, voice-driven interactive experience. The system listens, reasons, and executes actions across a containerized environment, acting as an AI co-pilot for self-healing infrastructure.

Primary Objective: Build a deployable, high-impact portfolio piece demonstrating mastery of agentic workflows, real-time voice streaming, and Kubernetes infrastructure tooling.

2. System Architecture

The architecture bridges low-latency audio transport with advanced tool-using LLM workflows.

LAYER	TECHNOLOGY STACK	RESPONSIBILITY
Orchestration	LangGraph, FastAPI	Manages conversational state, multi-agent routing, and function calling (tool execution).
Voice Transport	LiveKit, WebSockets	Handles full-duplex WebRTC streaming to maintain sub-second conversational latency.
AI Models	Whisper (STT), OpenAI/Anthropic, TTS	Transcribes speech, generates reasoning, and synthesizes natural voice responses.
Infrastructure	Kubernetes Python Client, Prometheus	Exposes cluster metrics and management APIs as deterministic tools for the agent.
Frontend	Next.js, Tailwind CSS	

LAYER	TECHNOLOGY STACK	RESPONSIBILITY
		Visualizes the audio waveform, cluster health, and real-time agent reasoning steps.

3. Core Features & "Wow" Factors

Feature 1: Conversational Telemetry

Instead of manually querying dashboards, the user asks plain-language questions via voice. The agent translates this into Prometheus PromQL queries or Kubernetes API calls.

- *User*: "What's the CPU usage on the payment gateway pods looking like?"
- *Agent*: (Queries Prometheus) "The payment pods are running slightly hot at 85% CPU. I noticed a spike in traffic over the last five minutes."

Feature 2: Agentic Remediation & Safety Checks

The agent does not just monitor; it takes action. However, it implements a crucial **Human-in-the-Loop (HITL)** safety mechanism for destructive actions.

- *Agent*: "Given the high CPU, would you like me to scale the replica set from 3 to 5?"
- *User*: "Yes, go ahead."
- *Agent*: (Executes K8s API call) "Done. The new pods are spinning up now."

Feature 3: The "War Room" Visualization

The frontend removes the black box of AI. While the user speaks, a terminal-style UI shows the agent's internal reasoning (e.g., [Tool: Querying Prometheus] -> [Reasoning: Traffic spike detected] -> [Generating Audio]). This proves the underlying engineering complexity visually.

4. Implementation Roadmap

Phase 1: Deterministic Tooling & Infrastructure Bridge

Establish the foundation. Write the Python functions that interact with the cluster. These functions must be robust, as they will become the tools the LLM invokes.

kubernetes-client

prometheus-api

- Create read-only tools: `get_pod_status()`, `get_node_metrics()`.
- Create action tools: `restart_pod()`, `scale_deployment()`.
- Set up local Minikube or kind cluster for safe testing.

Phase 2: LangGraph Orchestration

Wrap the tools in an agentic workflow capable of multi-step reasoning.

langchain

langgraph

- Define the state graph: `listen -> reason -> tool_use -> synthesize_response`.
- Implement memory so the agent remembers context (e.g., "Scale *that* deployment up").
- Test via text-based CLI to ensure orchestration logic is flawless before adding audio latency.

Phase 3: Real-Time Audio Transport

Integrate the voice layer. This is the most complex phase regarding latency optimization.

livekit

websockets

whisper

- Deploy a LiveKit room.
- Connect a continuous audio stream from the client microphone to the backend STT.
- Stream LLM text chunks directly into the TTS engine to minimize time-to-first-byte (TTFB) audio playback.

Phase 4: Frontend UI & Deployment

Build the recruiter-facing presentation layer and host the application.

next.js

docker

render/railway

- Build a sleek "Hold to Talk" interface with live terminal logs.
- Containerize the FastAPI backend and deploy alongside the Next.js frontend.
- Ensure the test Kubernetes cluster (acting as the "dummy" target for the agent) is safely sandboxed.